

WHAT IT IS

- A PyQt5 desktop plotting app that loads local numeric data files and renders interactive Matplotlib plots.
- The codebase centers on a MainWindow UI, an AppController state layer, and a PlotCanvas that draws configured curves and subplots.

WHO IT'S FOR

- Primary persona: a desktop user who needs to inspect or compare tabular numeric data by building custom 2D plots without writing plotting code.

WHAT IT DOES

- Loads local text-like data files and auto-detects delimiter/header structure before parsing numeric columns.
- Lets users add multiple curves, choose X/Y columns, rename curves, and assign them to primary or secondary Y axes.
- Supports subplot layouts, subplot assignment, and shared X/Y axis behavior.
- Applies plot styling controls including aspect ratio, color palettes, curve colors, tick density, legends, major grid, minor ticks, and minor grid.
- Persists full plot state as a JSON-based project file with data-file paths, plot config, and curve definitions.
- Reopens saved plot projects and skips curves whose referenced source files are missing.
- Includes Matplotlib navigation/customize tooling and syncs edited labels or line styling back into app state.

HOW IT WORKS

- Entry point: `pyqt_plotter_main.py` creates `QApplication` and `MainWindow`.
- UI layer: `MainWindow` builds the control panel, file/curve/project actions, subplot controls, and Matplotlib toolbar.
- Controller layer: `AppController` stores loaded `DataFile` objects, `Curve` definitions, and `PlotConfig`, then calls `PlotCanvas.draw_curves()`.
- Data layer: `DataFile.load_data_file()` reads local files, removes comments, detects delimiter, finds the first fully numeric row, and returns NumPy-backed columns.
- Rendering layer: `PlotCanvas` creates/reuses Matplotlib subplots, draws each curve, applies per-subplot/global config, and renders inside Qt.
- Persistence: `save_project/load_project` serialize app state to JSON (`.pproj` example found); external storage/services: Not found in repo.

HOW TO RUN

- 1. Use Python with the imported runtime libraries available: PyQt5, matplotlib, and numpy. Exact install command: Not found in repo.
- 2. From the repo root, run: `python pyqt_plotter_main.py`
- 3. In the app, add a data file, configure curves/settings, and optionally save a plot project.